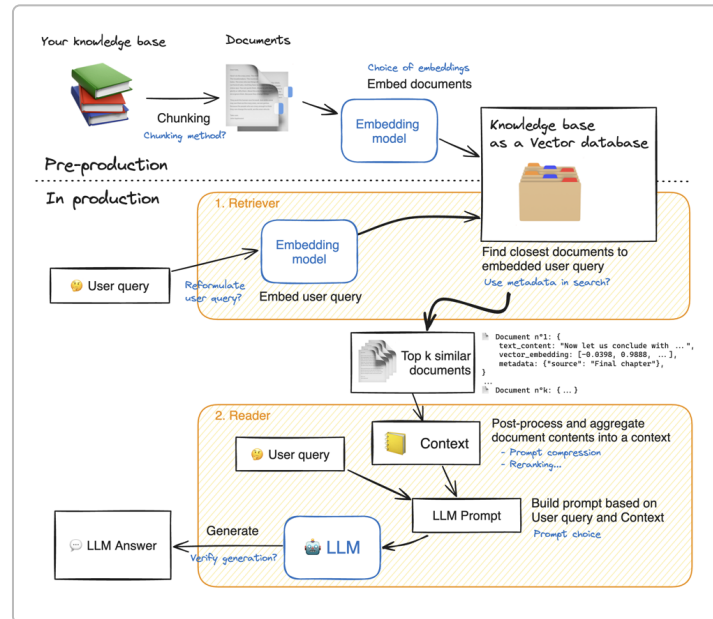
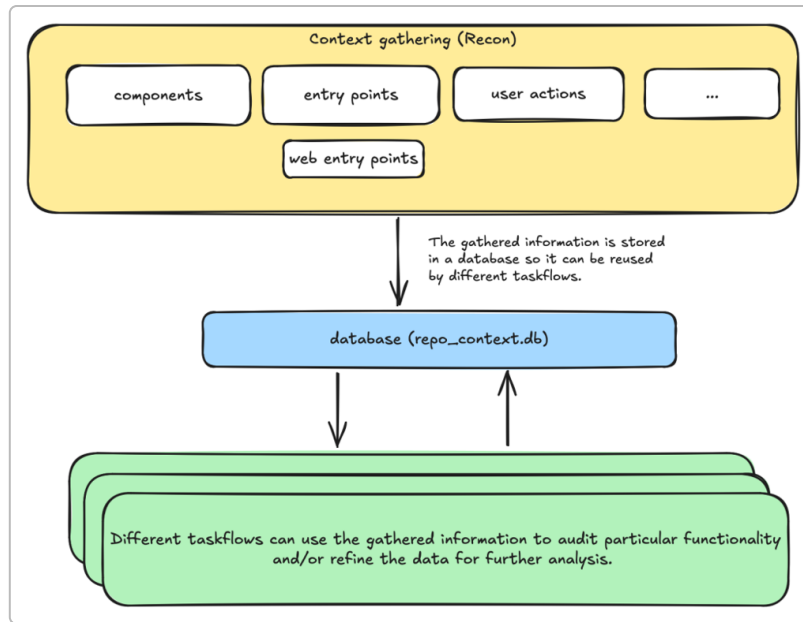


AI Autonomous Coding Agent



The system follows a **Retrieval-Augmented Generation (RAG)** architecture for code analysis. In the **indexing** phase, repository files are loaded and split into meaningful chunks; each chunk is passed through an LLM embedding model and stored in a vector database (e.g. Chroma) ¹. In the **query** phase, a user's request (e.g. "Find bugs in file X") is embedded and used to retrieve relevant code chunks. These chunks plus the original query form a prompt to an LLM to analyze or modify the code ². The diagram above illustrates this RAG workflow: code documents are indexed ahead of time, and at runtime the LLM retrieves context and generates a response ¹ ².



To prepare the repository for RAG, the system **scans the repo and gathers context**. For example, GitHub's Security Lab framework first breaks a repo into components and entry-points, storing this context in a database for reuse ³. Similarly, our agent will use the GitHub API to list all files (`GET /repos/{owner}/{repo}/git/trees/{branch}?recursive=1` ⁴) and read each file's contents. A background **worker** (via Redis+BullMQ) can automate this scanning. In our design, the scanner extracts metadata and code snippets (e.g. function definitions, data flows) and optionally stores high-level info in a SQL table or vector store. The image above shows a context-gathering phase: after scanning, all relevant code metadata is stored for downstream analysis ³.

Tech Stack & Architecture

- **Frontend (Next.js 14, React, TailwindCSS):** A Next.js 14 app (App Router) provides a dashboard UI. It includes pages for GitHub account connection, repo selection, and reviewing detected issues. Tailwind CSS is used for styling. In Next.js App Router, data fetching can often happen in Server Components (e.g. loading connected repos or issues directly from the backend) ⁵.
- **Backend (Node.js, Express, REST API):** An Express server provides APIs for GitHub actions, job management, and AI tasks. It communicates with the frontend via JSON. Key endpoints include: GitHub OAuth (storing a personal access token), repository scanning (trigger jobs), embedding retrieval, and pull-request creation. We recommend using Prisma (or a similar ORM) with PostgreSQL to store user info, connected repos, and task status ⁶.
- **Queue System (Redis + BullMQ):** Time-consuming tasks (repo scanning, embedding generation, fix generation) are offloaded to background workers using BullMQ queues on Redis. Redis is a high-speed in-memory store well-suited for queues ⁷. BullMQ provides reliable job queues with scheduling, retries, delays, and prioritization ⁸. For example, when the user initiates a scan, the API can enqueue a "scanRepo" job; a worker then performs scanning/embedding and reports results back to the database.

- **AI Layer (LangChain + LLM):** We integrate with an LLM (e.g. OpenAI GPT-4/5.2, Anthropic Claude) via LangChain. LangChain handles document loading, splitting, embedding, and chain-of-thought prompts. Although LangChain originated in Python, **LangChain.js** in Node is sufficiently mature for production use ⁹. We use LangChain's `RecursiveCharacterTextSplitter` (including language-aware splitting) to chunk code files ¹⁰. For example, one can create a splitter that respects Python or JavaScript syntax. Each chunk is sent to an embedding model (e.g. `text-embedding-ada-002`), and the resulting vector is stored in Chroma (an open-source vector DB) ¹¹. The Hugging Face RAG cookbook similarly recommends embedding all document chunks in a vector store for fast retrieval ¹ ².
- **Vector Database (Chroma):** We use ChromaDB for storing embeddings. (Installation: `pip install chromadb` or `npm install chromadb` ¹¹.) Chroma can persist embedding vectors and metadata, allowing fast k-NN retrieval. At query time, LangChain performs a vector search on Chroma to fetch the top-K relevant code chunks.

GitHub Integration

1. **Authentication:** Users supply a GitHub personal access token (PAT) on the frontend. The token is securely stored (e.g. encrypted in the database) and attached to API requests as `Authorization: Bearer <token>` (or `Authorization: token`) ¹². This PAT must have scopes for repository read/write and pull-request creation. The Express backend will include this header when calling GitHub's REST API.
2. **Repository Listing/Selection:** Once authenticated, the frontend calls the backend to list accessible repos. This can use `GET /user/repos` or `GET /orgs/:org/repos` via the GitHub API. The user picks a repository to scan.
3. **Creating a Branch:** When preparing fixes, the backend will create a new branch via the GitHub API. First it fetches the SHA of the base branch (e.g. `main`) by calling `GET /repos/{owner}/{repo}/git/refs/heads/main`. Then it **POSTs** to `https://api.github.com/repos/{owner}/{repo}/git/refs` with JSON `{"ref": "refs/heads/fix-xyz", "sha": "<base-commit-sha>"}` ¹³. This creates a new branch named `fix-xyz`.
4. **Committing Code:** To update or add a file, use the "Create or update file contents" endpoint. The backend sends a **PUT** request to `/repos/{owner}/{repo}/contents/{path}` with a JSON body containing:

```
{
  "message": "Fix bug in function foo()",
  "content": "<base64-encoded new file content>",
  "branch": "fix-xyz",
  "committer": {"name": "AI Agent", "email": "bot@example.com"}
}
```

GitHub will commit the new content to the `fix-xyz` branch. (See example in GitHub docs ¹⁴ ; on success it returns a 201 and commit SHA.)

1. **Opening a Pull Request:** After commits are made, a pull request is created by **POST**ing to `/repos/{owner}/{repo}/pulls` with `{ title, head, base, body }` in the JSON. For example, using Axios:

```
await axios.post(
  'https://api.github.com/repos/${owner}/${repo}/pulls',
  { title: prTitle, head: "fix-xyz", base: "main", body: prDescription },
  { headers: { Authorization: `token ${pat}`, Accept: 'application/vnd.github.v3+json' } }
);
```

This creates the PR for review ¹⁵ .

Repository Scanner

- **Cloning/Reading Files:** The system uses the GitHub API to enumerate and fetch files. One efficient method is using the **Git Trees** API: `GET /repos/{owner}/{repo}/git/trees/{branch}?recursive=1` returns all file paths and blob SHAs ⁴ . For each file path of type "blob", we then call `GET /repos/{owner}/{repo}/contents/{path}` (or use the blob URL) to retrieve the content. This avoids needing to clone the repo on disk.
- **Chunking:** All code files are split into chunks. For example, use LangChain's `RecursiveCharacterTextSplitter.from_language(...)` to split code by language semantics ¹⁰ . We choose chunk sizes (e.g. 1000–2000 tokens) and overlap (to preserve context) as recommended ¹⁶ . The result is an array of code segments with references to their file and position.
- **Embedding Ingestion:** Each chunk is embedded and stored. For example, in Python with LangChain:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from chromadb import Client
splitter = RecursiveCharacterTextSplitter(chunk_size=1500,
chunk_overlap=200)
docs = splitter.split_documents(code_documents)
embeddings = OpenAIEmbeddings() # or any model
chroma = Client()
for doc in docs:
    vector = embeddings.embed_query(doc.page_content)
    chroma.add(collection="codebase", ids=[doc.metadata["filename"]],
embeddings=[vector], metadatas=[doc.metadata])
```

(LangChain's Python tutorials show similar code splitting and embedding ¹⁶ ¹⁰ .)

Bug Detection Engine

- **LLM Analysis:** The core of bug detection is prompting the LLM. We design prompts that feed in relevant code context and ask the LLM to identify issues. For instance: “Review the following JavaScript function for syntax errors, security issues, or bad practices. List all problems and their locations.” We can include retrieved code chunks from the vector DB in the prompt to give the LLM enough context (RAG helps keep it in-window).

- **Example Prompt:**

```
You are a senior software engineer. Check the code below for bugs or
vulnerabilities:
```javascript

(chunk of code here)
```

List each issue found, its type (syntax, security, logic, style), and a brief explanation. ```

We may run this analysis iteratively on each file or component. The security auditing blog notes that LLM workflows should start with a **threat-modelling** step (gathering component roles, inputs, privilege boundaries) before looking for issues <sup>17</sup> . Our system similarly tags code contexts (e.g. “this function handles user input X”) and uses that context in prompts.

- **Effectiveness:** LLMs have shown surprising power for large-scale code auditing. For example, the BugStone system combined static analysis with an LLM to find **22,568** potential bugs in the Linux kernel (sampling 400, they confirmed 246 true issues) <sup>18</sup> . This indicates that AI-driven scans can surface many real defects at scale. We mitigate hallucinations by giving strict instructions and focusing on one component at a time.

## Code Fix Generator

When the bug detector identifies issues, the AI agent attempts to generate fixes. This can be another LLM prompt like:

```
The code above has a bug at line 42: [brief description]. Provide a corrected
version of the code snippet, with an explanation of the fix.
```

We use tools like GitHub Copilot or ChatGPT code models for edits. LangChain's agent tools or function calling can also structure this: e.g., a “CodeFixTool” that takes code and returns a fixed file text. Each suggested fix is accompanied by an explanation string. The frontend will display the original code, the issue description, and the AI-proposed patch side-by-side for user review.

## Automatic Pull Request Creator

Once fixes are generated, the backend automates PR creation:

1. **New Branch:** Create a branch named like `auto-fix-<timestamp>` using the Git Ref API <sup>13</sup>.
2. **Commit Changes:** For each file with a fix, call the GitHub “create/update file” API as shown above <sup>14</sup>. The commit message can include the issue ID or description.
3. **Open PR:** Finally, POST to `/repos/{owner}/{repo}/pulls` with `title` “Fix: [short summary]” and `body` explaining the fixes <sup>15</sup>.
4. **Approval Workflow:** The dashboard shows pending PRs. Users can review the AI’s diff and only click “Approve” when satisfied, after which the PR is officially opened on GitHub.

## Dashboard UI

The Next.js dashboard provides:

- A **login/connect page** to enter or authenticate the GitHub PAT.
- A **repo selector** UI showing user’s repos (fetched from `/user/repos`).
- A **Scan button** to start analysis on the selected repo.
- A **Results view** listing detected issues: each row has file path, line number, issue description, and a “Show fix” preview.
- A **Diff/Review** pane where the AI-generated code change is highlighted (using a diff library).
- An **Approve PR** action: once clicked, the backend runs the PR creation flow. All pages are styled with Tailwind for a clean, responsive interface.

## Autonomous Worker

To run continuously, a background **worker** polls or listens for new repos to audit. For example, on a schedule (via a cron-like job) or when a webhook triggers (e.g. new commit), the worker enqueues a “scan” job. Workers use BullMQ: they subscribe to a queue (e.g. `scanQueue`) and process jobs in parallel <sup>8</sup>. When a job is pulled, the worker executes the scanning + AI steps above, then stores the results.

Because BullMQ supports retries and concurrent workers <sup>8</sup>, we can scale the scanning (e.g., multiple repos at once) and ensure transient errors don’t lose tasks. Job statuses (queued, running, completed) can be stored in PostgreSQL for monitoring.

## Project Structure

A suggested folder layout:

```
frontend/ # Next.js 14 App Router dashboard
├─ app/ # pages and layouts (e.g. connect/, dashboard/, api/auth/)
├─ components/ # React UI components
├─ styles/ # Tailwind config, global CSS
└─ ...
```

```

backend/ # Express REST API
├ routes/ # GitHub and scan endpoints
├ controllers/ # Business logic for each route
├ services/ # GitHub API calls, AI orchestration
├ models/ # Prisma/Postgres ORM schema
├ workers/ # Job processors (BullMQ)
└ server.js # Express setup
ai-engine/ # Code analysis and RAG pipeline (can be Node or Python)
├ pipeline.js # Functions to split, embed, and query code
├ chat.js # LLM prompt utilities
└ ...
workers/ # Background jobs
├ scanWorker.js # Dequeue scan jobs, call AI engine
├ fixWorker.js # Dequeue fix-generation jobs
└ ...
docker-compose.yml # Sets up PostgreSQL, Redis, (optionally Chroma), and all
services
README.md # Overview, setup and architecture explanation

```

## Setup & Deployment

- **Environment:** We use Docker Compose to run all services locally: one container each for the Next.js frontend, Node backend, Redis, PostgreSQL, and (if needed) a Chroma server.
- **Configuration:** The README instructs creating a GitHub PAT and setting it as an environment variable or in a secure vault. Database schemas (users, repos, issues) can be managed via Prisma migrations.
- **Run:** After cloning, `docker compose up` builds and starts the stack. The web UI is available at `localhost:3000`.

**References:** This design follows modern RAG and microservice patterns. For example, LangChain tutorials outline the one-time indexing (loader → splitter → embed → vector store) and per-query retrieval steps <sup>1</sup> <sup>2</sup>. The GitHub API usage (auth via `Authorization` header <sup>12</sup>, refs for branches <sup>13</sup>, content commits <sup>14</sup>, PR creation <sup>15</sup>) enables automating code changes. Redis+BullMQ is a proven setup for background jobs <sup>8</sup> <sup>7</sup>. And recent research (e.g. BugStone) demonstrates that LLMs can effectively scan large codebases for bugs at scale <sup>18</sup>. Each module is implemented in a clean, modular fashion (separate routes, services, and workers) to ensure maintainability and production readiness.

<sup>1</sup> <sup>2</sup> <sup>10</sup> <sup>11</sup> <sup>16</sup> How to Build RAG Applications with LangChain

<https://oneuptime.com/blog/post/2026-01-26-langchain-rag-applications/view>

<sup>3</sup> <sup>17</sup> How to scan for vulnerabilities with GitHub Security Lab's open source AI-powered framework - The GitHub Blog

<https://github.blog/security/how-to-scan-for-vulnerabilities-with-github-security-labs-open-source-ai-powered-framework/>

<sup>4</sup> Get all file names from a Github repo through the Github API - Stack Overflow

<https://stackoverflow.com/questions/25022016/get-all-file-names-from-a-github-repo-through-the-github-api>

- 5 Next.js 14 App Router Project Structure: The Patterns That Actually Scale - DEV Community  
<https://dev.to/whoffagents/nextjs-14-app-router-project-structure-the-patterns-that-actually-scale-7a6>
- 6 7 Building a Full-Stack Email Job Scheduler with Next.js, Node.js, Redis, Prisma, and BullMQ - DEV Community  
<https://dev.to/torver213/building-a-full-stack-email-job-scheduler-with-nextjs-nodejs-redis-prisma-and-bullmq-38m3>
- 8 Scalable Background Processing in Node.js with BullMQ and Redis | by Bhargav Undhad | Medium  
<https://medium.com/@bhargavundhad.mscit22/scalable-background-processing-in-node-js-with-bullmq-and-redis-e55042d164d6>
- 9 I Built an AI Agent in Node.js Using LangChain — Here's What Actually Worked | by Ram Milan | CodeToDeploy | Medium  
<https://medium.com/codetodeploy/i-built-an-ai-agent-in-node-js-using-langchain-heres-what-actually-worked-bc612fde39e1>
- 12 Authenticating to the REST API - GitHub Docs  
<https://docs.github.com/en/rest/authentication/authenticating-to-the-rest-api>
- 13 git - Github API - create branch? - Stack Overflow  
<https://stackoverflow.com/questions/9506181/github-api-create-branch>
- 14 REST API endpoints for repository contents - GitHub Docs  
<https://docs.github.com/en/rest/repos/contents>
- 15 Automate Github Pull Requests With NodeJS API - DEV Community  
<https://dev.to/redbonzai/automate-github-pull-requests-with-nodejs-api-48gj>
- 18 One Bug, Hundreds Behind: LLMs for Large-Scale Bug Discovery  
<https://arxiv.org/html/2510.14036v1>